OSU Satellite TV for MITRE eCTF 2025

Contents

1. Team

The team is comprised of the following undergraduate students from the Ohio State State University:

- Jack Leverett
- Divij Sasidhar
- Mark Bundschuh
- Maxwell McGee
- Matthew Weikel
- Debjani Hill
- Aditya Gupta
- Jacob Mcquade
- Eva Smith
- William Comer
- Diego Noria
- Zihao Zhang
- Mason Erwine

- Christopher Begines
- Haoling Zhou
- Ryan Jackman
- Joshua Sims
- Rachel Thoi
- Cadan Keller
- James Vendetti

Along with advisors:

- Felix Engelmann
- Julia Armstrong

2. Introduction

This document describes OSU's implementation of a secure satellite TV system as specified by MITRE's eCTF 2025. The Satellite TV system features a simulated satellite sending ASCII art to subscribed hardware Decoder devices, implementing Functional Requirements 2.1. The decoder devices are MAX78000FTHR microcontrollers by Analog Devices. The system is secured according to the Security Requirements 2.2, and designed to defend against attacks as outlined in Attack Scenarios 2.3, which revolve around erroneously decoding frames sent by a satellite.

2.1. Functional Requirements

The two main components are the Encoder and Decoder. The encoder receives frames of up to 64 bytes, and converts them into our custom format before sending it over the satellite link. Then, each decoder receives the frame, and must decode the message into the original 64 bytes.

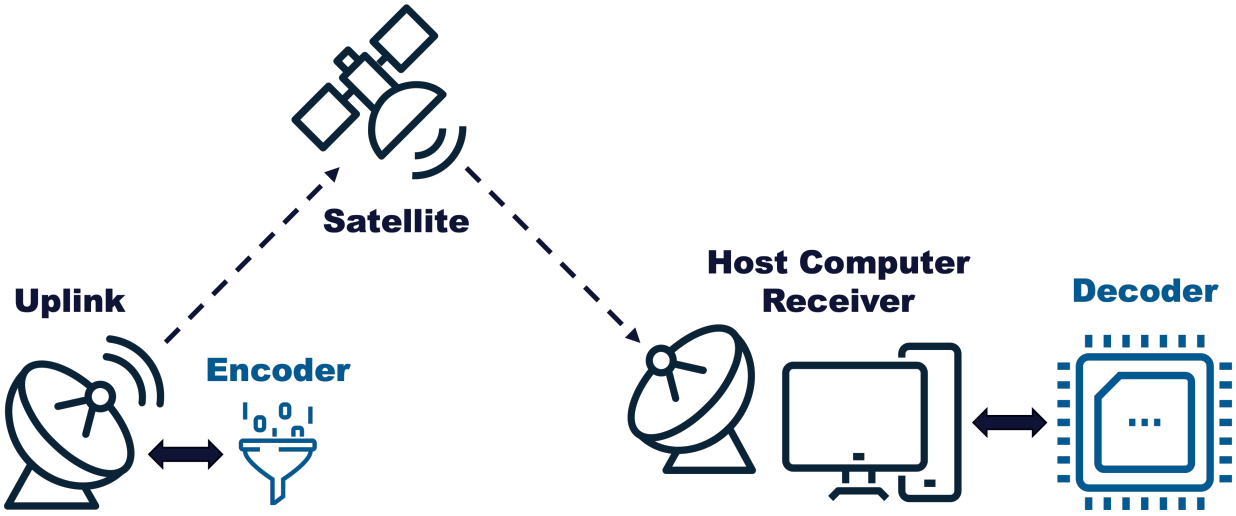

Figure 1: Full system with blue highlights indicating components which need to be implemented.

2.1.1. Functional Requirement 1: List Channels

"The Decoder must be able to list the channel numbers that the Decoder has a valid subscription for."

2.1.2. Functional Requirement 2: Update Subscription

"The Decoder must be able to update it's channel subscriptions with a valid update package. If a subscription update for a channel is received for a channel that already has a subscription, the Decoder must only use the latest subscription update."

2.1.3. Functional Requirement 3: Decode Frame

"The Decoder must properly decode TV frame data from any channel that has a valid and active subscription installed or for any emergency broadcast frames."

2.2. Security Requirements

2.2.1. Security Requirement 1: Active Subscription

"An attacker should not be able to decode TV frames without a Decoder that has a valid, active subscription to that channel."

2.2.2. Security Requirement 2: Authentication

"The Decoder should only decode valid TV frames generated by the Satellite System the Decoder was provisioned for."

2.2.3. Security Requirement 3: Increasing Timestamps

"The Decoder should only decode frames with strictly monotonically increasing timestamps."

2.3. Attack Scenarios

We consider 5 different scenarios where the attackers have access to certain information that the system is designed to protect against.

2.3.1. Attack Scenario 1: Expired Subscription

An attacker has physical access to a decoder with a valid subscription to a channel. The timestamp of the current frames sent by the encoder is past the ending timestamp of the attacker's subscription.

2.3.2. Attack Scenario 2: Pirated Subscription

An attacker has physical access to a decoder, and a valid subscription to a channel for a different decoder which they do not have physical access to.

2.3.3. Attack Scenario 3: No Subscription

An attacker has physical access to a decoder, but has no subscriptions.

2.3.4. Attack Scenario 4: Recording Playback

An attacker has physical access to a decoder, and has been recording the raw data sent by the satellite for some time. Then, they get a valid subscription for the channel. In other words, the attacker should not be able to read past frames given an active subscription.

2.3.5. Attack Scenario 5: Pesky Neighbor

An attacker has physical access to a decoder, and spoofs the satellite signal to a neighbor's decoder which has a valid subscription. The attacker does not have physical access to the neighbor's decoder.

3. Deployment

3.1. Secrets

The following secrets are generated for a deployment. Once generated, they are global and read only.

- 1 32 byte seed per encrypted channel $S_{\text{channel\_id}}$
- 1 64 byte subscription salt, $S_{\text{subscription}}$
- 1 Ed25519 asymmetric key pair, secret key SK and public key PK

3.2. Decoder

The firmware must be built for each separate decoder device. When building, specify the `DEVICE_ID` environment variable as a 4 byte hexadecimal number starting with `0x`.

4. System Design

4.1. Encode and Decode

- u8 channel ID
- u64 timestamp
- [64]u8 message
- [64]u8 signature

Each frame has an integer timestamp $t \in [0, 2^{64} - 1]$. Each timestamp has a unique symmetric key k_t which the encoder will use to encrypt the message, and the decoder will use to decrypt the message, both using Salsa20. The keys are derived with a *binary hash key derivation tree*. Let H be a cryptographic hash function. We have chosen $H = \text{blake3}$. Let L and R be salted hash functions $L(x) = H(x + \text{"left"})$ and $R(x) = H(x + \text{"right"})$. The encoder uses secret $S_{\text{channel\_id}}$ for the channel id currently being broadcasted, and generates a root hash $h = H(S)$. Then, two hashes $h_0 = L(h)$ and $h_1 = L(h)$ are derived. Then $h_{00} = L(h_0)$, $h_{01} = R(h_0)$, $h_{10} = L(h_1)$, and $h_{11} = R(h_1)$, and so on. This process continues to form a tree of height 64. The leaves of the tree form the keys k_t, one for every timestamp. Figure 2 shows a hash key derivation tree for $t \in [0, 7]$ and has 8 leaves. Note that the full sized tree has 2^{64} leaves, the leaves are computed on demand for particular timestamps since it would be computationally infeasible to precompute all keys for such a large tree.

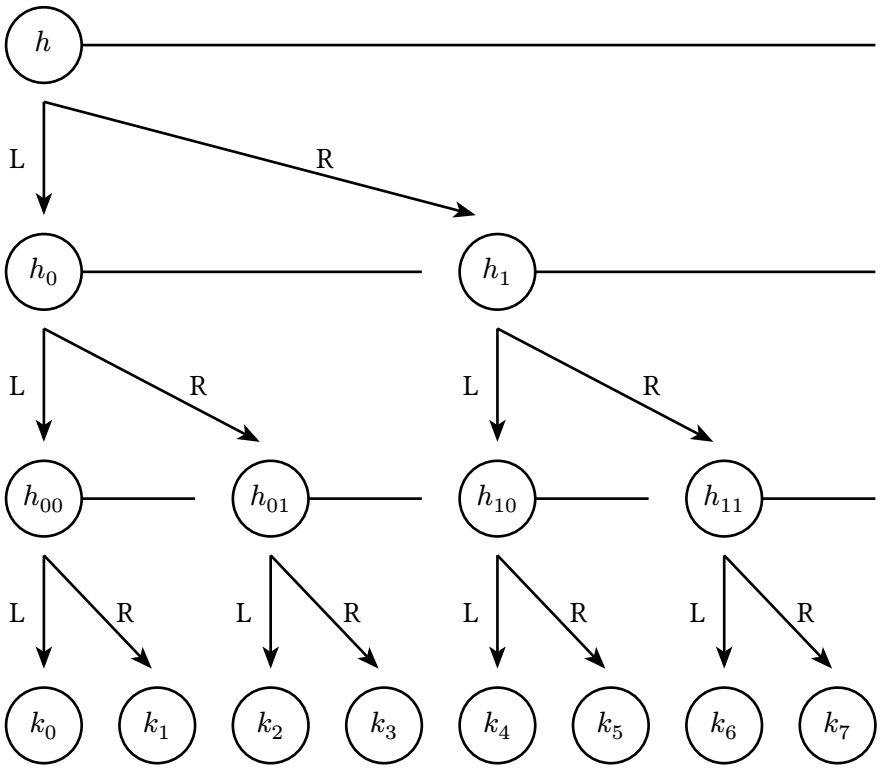

Figure 2: A hash key derivation tree from $t = 0$ to $t = 7$

To send the keys to the decoder, they are compressed into a subscription file. The compression is done by finding the largest power of two intervals which span the entire interval of timestamps $t \in [\text{start}, \text{end}]$. For example, all keys in $[1, 5]$ can be derived by $h_{001} \Rightarrow \{k_1\}$, $h_{01} \Rightarrow \{k_2, k_3\}$, and $h_{10} \Rightarrow \{k_4, k_5\}$, so only h_{001}, h_{01}, and h_{10} are needed to form a subscription. The worst case interval is $[1, 2^{64} - 2]$ which requires 126 subtree roots.

- **SR1**
- **AS1**: Expired Subscription
- **AS3**: No Subscription
- **AS4**: Recording Playback

4.2. Subscription

- u8 channel ID
- u64 start timestamp
- u64 end timestamp
- [up to 126][16]u8 packed subtree root hashes

The decoder runs the same algorithm using the start and end timestamps as the encoder to figure out where the subtree root hashes are, so no extra metadata is required to positions the hashes, or to know how many there are.

The entire subscription payload is symmetricly encrypted using Salsa20 with a key $k_{\text{subscription}} = H(S + \text{DEVICE\_ID})$. $k_{\text{subscription}}$ is stored within the decoder when it is provisioned and can be used to decrypt a subscription. This ensures that subscriptions can only be used by the decoder the subscription was generated for.

- **AS2**: Pirated Subscription

4.3. Signatures

The encoder uses the secret key SK to sign every message with a 64 byte signature. The public key PK is stored within all decoders and verifies every message to ensure it came from the encoder it was provisioned with.

- **SR2**
- **AS5**: Pesky Neighbor

4.4. Global Timestamp

A global timestamp is kept in the RAM of the decoder, and is used to reject misordered frames. It increases on every successful decode.

- **SR3**

4.5. Zig

The Zig programming language was selected as the language to write the secure decoder in. Zig has memory safety features like protection from indexing an array out of bounds, double free protections and more. Additionally, it includes a comprehensive standard library which securely implements many cryptographic functions, among many other quality of life functions. Most importantly, Zig can do all this while maintaining perfect C compatibility and using the MSDK provided by Analog Devices Inc (the developer of the MAX78000FTHR), without requiring a rewrite of the Hardware Abstraction Layer (HAL). This allowed the team to move incredibly quickly while ensuring a high level of security.